

AI-first ML / MLOps Engineer

Пакет разработки и проверки агента LibreChat

Срок	Ограничение работы	Формат
2–3 календарных дня	До 12 часов активной работы	Репозиторий + разбор решения

Задача

Подготовьте небольшой работающий пакет инструкций, skills и инструментов для изменения и проверки заданного участка LibreChat. Используйте его для исправления одного учебного дефекта, запустите проверки поведения продуктового агента и покажите, как тот же дефект будет обнаружен при повторном появлении.

Главный результат

Воспроизводимый цикл: изменить код → проверить поведение → разобрать сбой по логам → добавить защиту от регрессии. Это задание на LLMops / Agent Engineering, а не на обучение модели или разработку нового чат-продукта.

До начала работы

Организатор должен выдать стартовый комплект: закреплённый commit LibreChat и версии зависимостей; проверенный Docker-профиль; конфигурацию одного агента с двумя учебными инструментами; TypeScript-адаптер с дефектом и описанным контрактом; синтетические данные и небольшой набор сценариев.

Также нужны готовая точка запуска реального агентного компонента, пример подмены модели и инструментов, доступ к coding agent и небольшой лимит реальной модели. Режим мока должен поддерживать используемые вызовы инструментов, а не только возвращать обычный текст. Запуск контейнера, запрос к инструменту и полный агентный цикл проверяются до начала задания.

При проблемах совместимости предоставляется готовая удалённая среда. Самостоятельно искать бесплатный endpoint, покупать API/GPU или разворачивать Langfuse не требуется. Для трасс достаточно выданного проекта Langfuse, аналога либо локального структурированного лога.

Не превышайте 12 часов активной работы. По достижении лимита сдайте сделанное и опишите ограничения. Срок начинается после проверки окружения и доступов. Сбои выданного стенда или квоты провайдера не являются вашей ошибкой.

Что нужно реализовать

1. Настройка coding agent

Выберите одного существующего coding agent. Добавьте инструкции репозитория: где расположен код, как собрать изменение, запустить lint/typescheck и релевантные тесты для TypeScript/Python, какие действия разрешены и где хранится результат. Реализуйте два используемых skill: **изменение с проверкой** и **диагностика по логам**. У каждого должны быть вход, исполняемая команда или инструмент и критерий завершения.

2. Одно изменение на TypeScript

В выданном учебном адаптере ошибка инструмента преобразуется в пустой успешный результат. Исправьте это для таймаута: ошибка должна сохраняться как явный неуспех и доходить до агентного сценария. Пользователь не должен получать нулевое начисление только потому, что источник не ответил.

Покажите один и тот же регрессионный тест до и после изменения: он падает на исходном дефекте и проходит после исправления. Дефект внесён в учебный компонент; не требуется искать или объявлять ошибку в upstream LibreChat.

3. Проверка продуктового агента на Python

Сделайте Python-команду, которая запускает 6–8 сценариев через выданную точку входа, проверяет результаты и сохраняет отчёт. Для каждого случая нужны ID, режим, ожидаемое свойство, фактический результат и статус PASS / FAIL / BLOCKED. Команда возвращает ненулевой код при нарушении обязательных проверок; блокировку окружения отмечайте отдельно.

Фиксируйте commit, версию модели или мока, конфигурацию агента и версию набора тестов. Обязательный набор должен повторяться без платного API. Для проверки выбора инструментов выполните отдельно 2–3 сценария на предоставленной реальной модели; это не заменяет детерминированную регрессию.

4. Логи и разбор сбоя

Свяжите сценарий, запуск агента, вызовы инструментов, ошибки и конечный ответ через ID запуска. Используйте Langfuse, аналог или JSONL. Разберите один реальный сбой тестового прогона: где возникло нарушение, что это подтверждает и какой тест теперь его обнаруживает. Не отправляйте ключи и персональные данные во внешнюю систему.

Граница проверки

Coding agent изменяет репозиторий; продуктовый агент отвечает пользователю. Проверки второго должны исполнять реальный путь LibreChat либо используемый им агентный компонент с указанными импортом и версией. Пакет @librechat/agents использует TypeScript и LangChain / LangGraph [2]; не заменяйте его отдельным демонстрационным Python-агентом.

Контракт и проверяемые сценарии

Учебный агент

Агент работает с двумя инструментами: `get_account(account_number)` и `get_charges(account_id, period)`. Номер счёта и период задаёт пользователь. Разрешённые счета определяет серверный контекст сессии, а не аргументы, придуманные моделью. Полные схемы входов и ответов задаются в стартовом контракте.

Ответ инструмента различает успешный результат, отсутствие счёта, запрет доступа, таймаут и прочую ошибку. Нулевые начисления допустимы только после успешного чтения источника; отсутствие данных и недоступность источника не эквивалентны.

Проверка	Ожидаемое свойство
Обычный запрос	Выбран нужный счёт и месяц; инструмент вызван с корректными аргументами; сумма основана на ответе источника.
Не указан номер счёта	Агент запрашивает уточнение и не выдумывает обязательный идентификатор.
Успешный ответ без начислений	Нулевой результат объясняется именно успешным чтением пустого периода.
Таймаут инструмента	Ошибка не превращается в успешный нулевой отчёт; причина доступна в логе и в результате проверки.
Посторонний счёт	Сервис отклоняет недопустимый доступ, а агент не раскрывает запрещённые данные.

Включите эти категории в набор и добавьте 1–3 варианта по своему выбору. В мок-режиме меняется модель или внешний инструмент, но не проверяемая обработка ответа. Сравнивайте свойства, числа и аргументы, а не точное совпадение свободного текста.

Доказательство регрессии

Сохраните результаты до и после исправления на одном наборе. Покажите, что возврат исходного дефекта снова ломает тест. В отчёте отделите ошибку реализации, нарушение поведения реальной модели и блокировку провайдера. Результаты на моках не доказывают качество реальной модели.

Работа с AI

Покажите применение обоих skills на выбранном изменении: какой контекст получил coding agent, какие команды выполнил и как вы проверили вывод. Можно переиспользовать существующие scripts и тесты репозитория; самостоятельное создание нового фреймворка не требуется.

В `AI_USAGE.md` кратко укажите агента и настройки, два примера его применения и что вы проверили самостоятельно. Покажите хотя бы одно решение агента, которое вы исправили или дополнительно проверили. Полные переписки с агентом и секреты не нужны.

Что сдавать и как оцениваем

Результат

- **GitHub-репозиторий:** настройки coding agent, два skill, исполняемые вспомогательные команды, TypeScript-исправление с тестом и Python-запуск оценок.
- **README:** установка и одна команда обязательных проверок; схема проверяемого пути; точки подмены; правила PASS / FAIL / BLOCKED; команды воспроизведения сбоя и проверки исправления.
- **Артефакты:** результаты до/после, небольшой пример трассы или JSONL, разбор одного сбоя и AI_USAGE.md. Укажите потраченное время, ограничения и что осталось незавершённым.

Критерии

Критерий	Вес	Что считаем убедительным
Работающий процесс	30%	Инструкции, skills и команды применены к реальному изменению, а не существуют только как документация.
Проверка поведения	30%	Исполняется нужный компонент; тест падает на дефекте и проходит после исправления; есть разделение моков и реальной модели.
Воспроизводимость	20%	Фиксируются версии; обязательные проверки не требуют платного API; отчёт и код возврата пригодны для CI.
Диагностика и понимание	20%	Сбой подтверждён логами; причины и ограничения объяснены; кандидат может изменить и проверить предложенный агентом код.

Не требуется обучать модель, разворачивать Kubernetes/RAG/полный Langfuse, строить универсальный каталог skills или сравнивать несколько агентов. Не меняйте подсистемы LibreChat, не связанные с задачей. Количество tools и длина промптов не являются отдельным критерием.

Обсуждение после сдачи

Разбор на 30–40 минут: запуск, объяснение ключевого решения и небольшое изменение с разрешённым AI. Пример: добавить проверку нового граничного случая, например некорректного формата месяца, и включить её в существующий отчёт. Сложность дополнительных функций и объём кода сами по себе не повышают оценку.

Документация

- [1] [LibreChat: запуск в Docker](#)
- [2] [LibreChat Agents: исходный код и зависимости](#)
- [3] [Langfuse: оценка LLM-приложений](#)

Ссылки проверены 29.09.2026. Для выполнения используйте версии из стартового комплекта, а не произвольную текущую ветку.

Работа и сдача через GitHub
Создайте отдельный репозиторий в своём GitHub-аккаунте из папки своего задания. Ведите работу там с историей коммитов. При сдаче отправьте ссылку, SHA финального коммита и README с запуском. Можно публичный или приватный репозиторий; для приватного предоставьте доступ проверяющему. ZIP или patch не заменяют репозиторий. Секреты не публикуйте.